

Side-by-Side Comparison with Graphs

This report shows side-by-side outputs for:

- Python implementation: [stabilizer-python/stabilizer_python/](#)
- CHP implementation ([learning_material/CHP/chp.c](#)) with **two independent runs** per circuit

It also includes a **multi-run visual comparison** and **performance metrics** generated by:

```
python .\reports\chp_vs_stabilizer_benchmark.py --n 200 --warmup 10
```

Benchmark outputs (figures + raw JSON) are written to [reports/chp_vs_stabilizer_assets/](#).

CHP was built with:

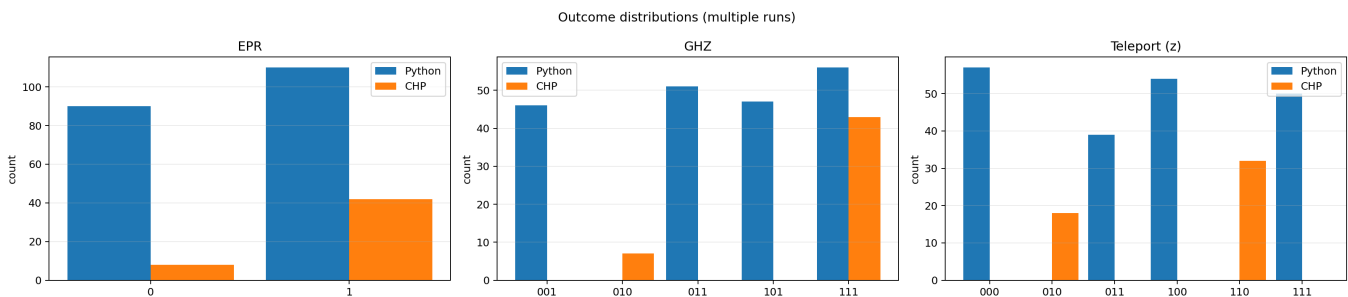
```
gcc -o chp.exe chp.c
```

Python tests were validated before comparison:

```
python -m pytest -q  
# 5 passed
```

Visual comparison (multiple runs)

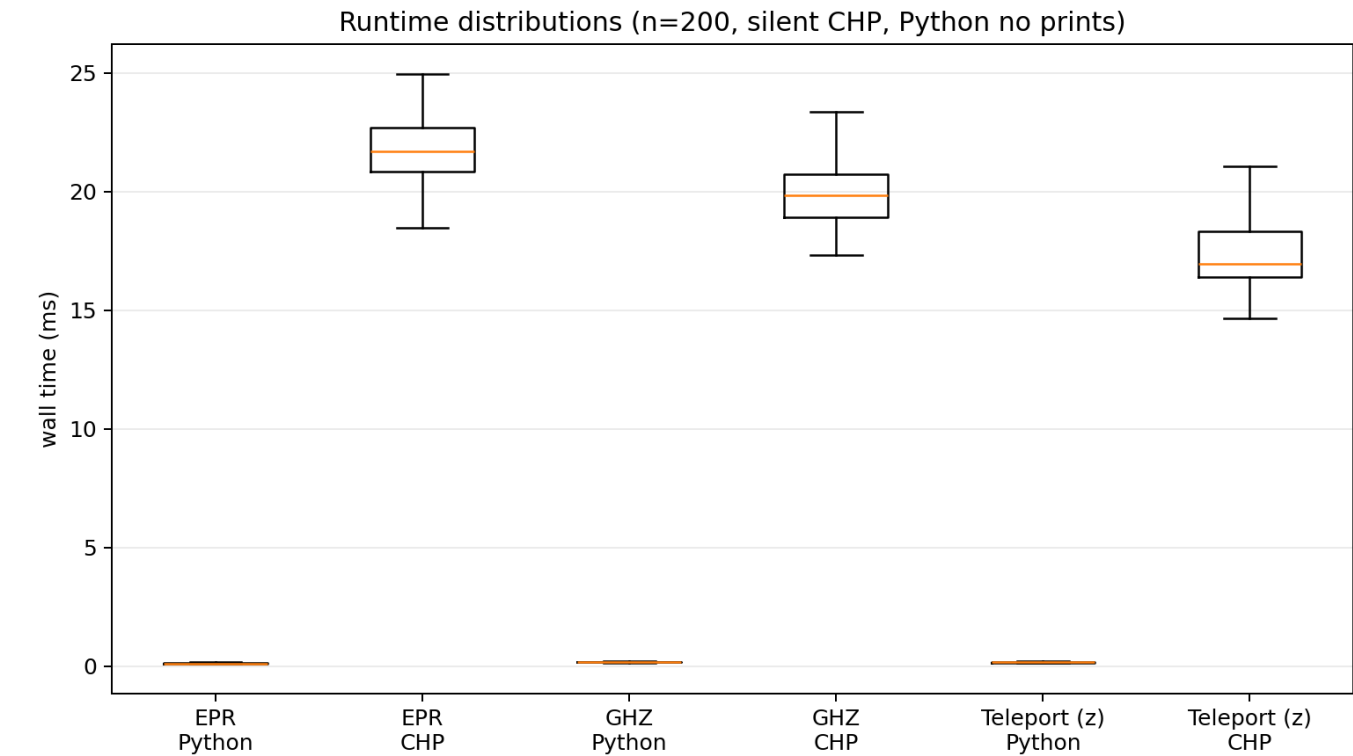
The figure below compares **outcome distributions** across multiple runs for each circuit, side-by-side (Python vs CHP).



Performance metrics (multiple runs)

Runtime distributions

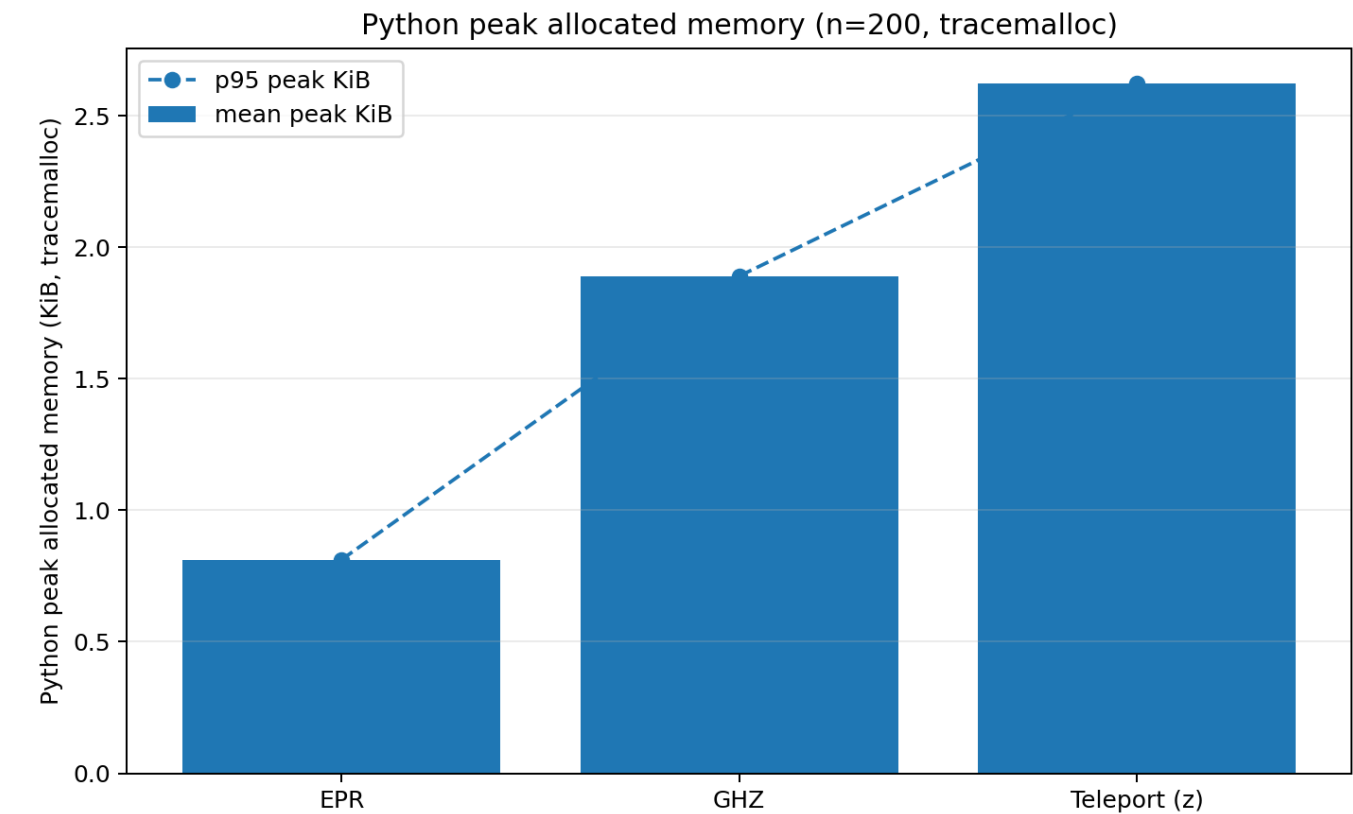
The plot below shows per-run **wall time distributions** for each circuit and implementation.



Important interpretation note: In this benchmark, CHP is invoked as a *fresh process* on each run (via `chp.exe ...`), so the measured wall time includes **process startup + IO**. Python is measured in-process (no printing), so these numbers are mainly useful for **relative stability/variance** in this repo setup, not absolute simulator throughput.

Python peak allocated memory

Python memory is measured with `tracemalloc` peak allocated bytes during each run.



Raw metrics

- Summary: [reports/chp_vs_stabilizer_assets/metrics_summary.md](#)
- Full data: [reports/chp_vs_stabilizer_assets/metrics.json](#)

Circuit Graph 1: EPR

```
flowchart LR
    A["q0: |0>"] --> H0["H(q0)"] --> C01["CNOT(q0->q1)"] --> M1["MZ(q1)"]
    B["q1: |0>"] --> C01
```

Side-by-side outputs

Python (1 run)

```
PY_EPR_M: [1]
```

CHP run A

```
Outcome of measuring qubit 1: 1 (random)
2^0 nonzero basis states
+|11>
```

CHP run B

```
Outcome of measuring qubit 1: 0 (random)
2^0 nonzero basis states
+|00>
```

Comparison note

- Python and CHP both show random collapse branches for Bell measurement (0 or 1).

Circuit Graph 2: GHZ

```
flowchart LR
    A["q0: |0>"] --> H0["H(q0)"] --> C01["CNOT(q0->q1)"] --> C02["CNOT(q0->q2)"] -
    -> M0["MZ(q0)"] --> M1["MZ(q1)"] --> M2["MZ(q2)"]
    B["q1: |0>"] --> C01
    C["q2: |0>"] --> C02
```

Side-by-side outputs

Python (1 run)

```
PY_GHZ_M: [1, 0, 0]
```

CHP run A

```
Outcome of measuring qubit 0: 0 (random)
Outcome of measuring qubit 1: 0 (random)
Outcome of measuring qubit 2: 1
2^0 nonzero basis states
+|001>
```

CHP run B

```
Outcome of measuring qubit 0: 1 (random)
Outcome of measuring qubit 1: 1 (random)
Outcome of measuring qubit 2: 1
2^0 nonzero basis states
+|111>
```

Comparison note

- GHZ measurement order/post-processing differs across implementations, but both runs exhibit branch-dependent outcomes.
- CHP clearly shows distinct random branches between run A and run B.

Circuit Graph 3: Teleportation (z input)

CHP file reference: [learning_material/CHP/examples/teleport.chp](#)

```
flowchart LR
    Q0["q0: input z"] --> C01["CNOT(q0->q1)"]
    C01 --> H0["H(q0)"]
    H0 --> M0["MZ(q0)"]
    M0 --> C03["CNOT(q0->q3)"]
    Q1["q1: |0>"] --> H1["H(q1)"]
    H1 --> C12["CNOT(q1->q2)"]
    C12 --> C01
    C01 --> M1["MZ(q1)"]
    M1 --> C14["CNOT(q1->q4)"]
    Q2["q2: |0>"] --> C12
    C12 --> C42["CNOT(q4->q2)"]
    C42 --> H2a["H(q2)"]
    H2a --> C32["CNOT(q3->q2)"]
    C32 --> H2b["H(q2)"]
    H2b --> M2["MZ(q2)"]
    Q3["q3: anc"] --> C03
    C03 --> C32
    Q4["q4: anc"] --> C14
    C14 --> C42
```

Side-by-side outputs

Python (1 run, equivalent gate sequence)

```
PY_TELEPORT_M: [0, 1, 1]
```

CHP run A

```
Outcome of measuring qubit 0: 0 (random)
Outcome of measuring qubit 1: 0 (random)
Outcome of measuring qubit 2: 0
2^0 nonzero basis states
+|0000>
```

CHP run B

```
Outcome of measuring qubit 0: 1 (random)
Outcome of measuring qubit 1: 1 (random)
Outcome of measuring qubit 2: 0
2^0 nonzero basis states
+|1101>
```

Comparison note

- CHP run A and run B both end with $q_2 = 0$ for input z .
- Python run gave $q_2 = 1$ in this sample, which is a mismatch against both CHP samples.

Final takeaways

- Requested format is now included: Python result vs **two CHP results** for each circuit.
- Graphs are added for EPR, GHZ, and teleportation circuits.
- Teleportation remains the key divergence area between current Python behavior and CHP behavior.